Universidad Católica del Norte
Facultad de Ingeniería y Ciencias Geológicas
Departamento de Ingeniería de Sistemas y Computación

Informe de taller N°4 Programación Avanzada:

Sistema de ofertas académicas UCN

Grupo 5:

Nombre: Vicente Cerda, Rut:20.844.038-1, francisco.barraza@alumnos.ucn.cl

Nombre: Francisco Barraza, Rut:21.830.784-1, vicente.cerda01@alumnos.ucn.cl

Profesor: Tomás Reimann

Ayudantes: Nicolás González, Cristian Jiménez, Diego Cortés

Rodrigo Aguilera, Samuel Fuentes

Fecha: 08/06/2025

Universidad Católica del Norte
Facultad de Ingeniería y Ciencias Geológicas
Departamento de Ingeniería de Sistemas y Computación

Taller 4 PA

Contexto:

En la Universidad católica del Norte se desea implementar un sistema para la automatización del proceso de creación y postulación a ofertas académicas, cosa que anteriormente se realizaba de forma manual y dispersa lo que conlleva a tiempos de espera más prolongados.

Solución:

Desarrollar un sistema que permita almacenar y gestionar ofertas académicas de tipo ayudantía, capstone y practica pre-profesional con el objetivo de automatizar la creación y en la postulación de ofertas con el objetivo de reducir el tiempo que tarda en realizar una oferta.

Cantidad de horas dedicadas a la realización del taller:

- Tiempo de diseño:
  -Francisco Barraza: 16 horas
  -Vicente Cerda: 1:00 horas

- Tiempo de codificar
  -Francisco Barraza: 18 horas
  -Vicente Cerda: 1:00 horas

- Tiempo de escribir el informe:
  -Francisco Barraza: 5:20 horas
  -Vicente Cerda: 1:00 hora

Estructura del programa

Se utilizan listas con nexo simple para los usuarios que representa la persona quien va a utilizar el programa y listas con nexo doble para las ofertas que contiene características únicas como en común entre los 3 tipos de ofertas de ayudantía, capstone y practica pre-profesional. Contenedores de listas simple como doble tanto de usuario como de la oferta, agregar y obtener tanto la posición como el usuario y para el contenedor de nexo doble agregar y eliminar.

Se realizan 2 lecturas de archivo, uno en la clase SistemaUsuarioArreglo donde se lee un archivo llamado “Usuarios.txt” el cual se leen datos iniciales del usuario y otro en la clase SistemaOfertaArreglo donde se lee un archivo llamado “OfertasAcademicas.txt que contiene información de las ofertas que hay disponibles.

```java
private void lecturaArchivo() throws IOException {  1 usage  ReimannHeiliger *
    Registro linea;
    ArchivoEntrada archivo = new ArchivoEntrada("Usuarios.txt");

    while (!archivo.isEndFile())
    {
        linea = archivo.getRegistro();

        String nombre = linea.getString();
        String rut = linea.getString();
        String correo = linea.getString();
        String contrasenia = linea.getString();

        this.contenedorNexoSimple.agregarUsuario(new Usuario(nombre,rut,correo,contrasenia));
    }
    archivo.close();
}
```

```java
private void lecturaArchivo() throws IOException {  1 usage  ReimannHeiliger *
    Registro linea;
    ArchivoEntrada archivo = new ArchivoEntrada("OfertasAcademicas.txt");
    Oferta oferta = null;

    // saltar la primera linea
    archivo.getRegistro();

    while (!archivo.isEndFile())
    {
        linea = archivo.getRegistro();

        char tipoOferta = linea.getChar();

        int id = Integer.parseInt(linea.getString());
        String titulo = linea.getString();
        String descripcion = linea.getString();
        String nombreUnidad = linea.getString();
        int duracion = linea.getInt();

        if (tipoOferta == 'A') {
            String nombreAsignatura = linea.getString();
            String rolAyudante = linea.getString();
            int cantidadHorasSemanalMensual = linea.getInt();
            float promedioMinimo = (float) linea.getDouble();

            oferta = new OfertaAyudantia(id,titulo,descripcion,nombreUnidad,duracion,nombreAsignatura,rolAyudante,cantidadHorasSemanalMensual,promedioMinimo);
        }

        else if (tipoOferta == 'C' || tipoOferta == 'P') {
            LocalDate fechaInicio = ValidadorFecha.convertirFecha(linea.getString());
            String empresaSolicitante = linea.getString();
            String guiaEmpresa = linea.getString();

            if (tipoOferta == 'C') {
                String tipoCapstone = linea.getString();
                int duracionEnMeses = linea.getInt();
                String[] carrerasNecesarias = this.procesarOcupaciones(linea.getString());
                int cantidadMinimaEstudiantes = linea.getInt();

                oferta = new OfertaCapstone(id, titulo, descripcion, nombreUnidad, duracion, fechaInicio, empresaSolicitante, guiaEmpresa, tipoCapstone, duracionEnMeses, carrerasNecesarias, cantidadMinimaEstudiantes);
            }
            else {
                String tieneRemuneracion = linea.getString();
                boolean remuneracion = tieneRemuneracion.equalsIgnoreCase( anotherString: "Si");

                oferta = new OfertaPractica(id, titulo, descripcion, nombreUnidad, duracion, fechaInicio, empresaSolicitante, guiaEmpresa, remuneracion);
            }
        }
        if (oferta != null) {
            this.contenedorNexoDoble.agregarOferta(oferta);
            oferta = null;
        }
```

Para mantener un orden en la implementación del programa se utiliza una estructura de paquetes los cuales son: dominio, servicios, utilidades y vista.

Dominio contiene las clases tanto del usuario como de las ofertas y sus contenedores. Ofertas hereda características que los 3 tipo de ofertas tienen, la clase oferta Actividad hereda de oferta pero que a su vez hereda características en común que solo tiene el tipo de oferta de capstone y practica pre-profesional. Y sus contenedores tanto de usuario como de la oferta.

Adicionalmente, al dominio creamos el paquete llamado colecciones cuyas colecciones contienen las listas enlazadas que utilizaremos para trabajar con los usuarios y ofertas. Listas con nexo simple para los usuarios y listas con nexo doble para las ofertas.

Servicios contiene la interface sistema que se encarga de llamar a los métodos y dando por parámetros lo que se va a usar, y una clase que implemente el sistema para desarrollar la lógica de los métodos como de la lectura del archivo.

Utilidades contiene clases que se encargan de validar datos que provoquen excepciones como ValidadorEntrada que se encarga de capturar una excepción si se ingresan letras a datos que requieran de números como enteros. ValidadorRut que se encarga de verificar si el número de caracteres que contiene un rut se cumplan y un ValidadorFecha que se encarga de verificar si la fecha extiende del plazo de duración.

Vista que contiene la clase VistaConsola que se encarga de todo el apartado visual del programa y de la interacción del usuario con el sistema.




```
}

public NodoDoble getCola() {  no usages  new *
    return cola;
}

public void setCabecera(NodoDoble cabecera) {  no usages  new *
    this.cabecera = cabecera;
}

public void setCola(NodoDoble cola) {  no usages  new *
    this.cola = cola;
}

public void agregarOferta(Oferta oferta){  4 usages  new *

    NodoDoble nuevo;

    try {
        nuevo = new NodoDoble(oferta);
    }catch (IllegalArgumentException ex){
        throw ex;

    }

    if(this.cabecera == null){

        this.cabecera = nuevo;
        this.cola = nuevo;

    }else{
        nuevo.setSiguiente(this.cabecera);
        this.cabecera.setAnterior(nuevo);
        this.cabecera = nuevo;
    }
    this.tamanio++;
}

public boolean eliminar(Oferta oferta){  1 usage  new *
```

Métodos del sistema

Registrarse

El método registrarse contiene 2 apartados: el apartado de SistemaUsuarioArreglo donde se realizan las validaciones como de longitud de caracteres, el rut sea único y que no se repita. Si cumple con las condiciones se agregará a la nueva persona al contenedor.

```
@Override  1 usage  ReimannHeiliger *
public String registrarse(String nombre, String rut, String correo, String contrasenia) {

    if (!ValidadorRUT.verificarRUT(rut)) {
        return "El rut" + rut + " no es valido";
    }
    //implememtar con contenedor nexo simple
    if (this.contenedorNexoSimple.obtenerPosicion(rut) != -1) {
        return "El usuario con el rut " + rut + " ya existe";
    }
    if (nombre.length() < 3 || nombre.length() > 255) {
        return "El nombre no puede tener menos de 3 caracteres o mas de 255.";
    }
    if (contrasenia.length() < 6 || contrasenia.length() > 255) {
        return "La contrasenia no puede tener menos de 6 caracteres o mas de 255.";
    }
    try {
        this.contenedorNexoSimple.agregarUsuario(new Usuario(nombre,rut,correo,contrasenia));
        return "El usuario con el rut " + rut + " ha sido registrado con exito";
    } catch (Exception e) {
        return e.getMessage();
    }
}
```

Y el de la vista consola donde se ingresan los datos como el nombre, rut, correo y contraseña de la persona por consola.

```
private void registrarse() {  1 usage  ReimannHeiliger *
    String nombreUsuario, rut, correo, contrasenia;

    StdOut.println(":::Registro:::");
    StdOut.println("Ingrese el nombre del usuario:");
    nombreUsuario = StdIn.readLine();
    StdOut.println("Ingrese el rut:");
    rut = StdIn.readLine();
    StdOut.println("Ingrese el correo:");
    correo = StdIn.readLine();
    StdOut.println("Ingrese la contrasenia:");
    contrasenia = StdIn.readLine();

    StdOut.println(this.sistemaUsuario.registrarse(nombreUsuario,rut,correo,contrasenia));
}
```

Iniciar Sesión

El método iniciarSesion contiene 2 apartados: el apartado de SistemaUsuarioArreglo donde se valida la posición de la persona en base al rut. Si la posición es igual a -1 significa que el rut de la persona no ha sido encontrado. Si la contraseña del usuario es distinta a la contraseña dada por parámetro retorna false. Si ocurre lo contrario significa que el usuario es verdadero por lo que retorna true y accede al menú.

```
@Override  1 usage  ± ReimannHeiliger *
public boolean iniciarSesion(String rut, String contrasenia) {

    if (!ValidadorRUT.verificarRUT(rut)) {
        throw new IllegalArgumentException("El rut" + rut + " no es valido");
    }
    int posicion = this.contenedorNexoSimple.obtenerPosicion(rut);

    if(posicion == -1){
        throw new IllegalArgumentException("No se encuentra el usuario con rut "+rut);
    }
    if(this.contenedorNexoSimple.obtenerUsuario(posicion) == null){
        throw new IllegalArgumentException("Usuario con el rut "+rut+" no existe");
    }
    if(this.contenedorNexoSimple.obtenerUsuario(posicion).getRut().equals(rut) && this.contenedorNexoSimple.obtenerUsuario(posicion).getContrasenia().equals(contrasenia)){
        return true;
    }
    Usuario usuario = this.contenedorNexoSimple.obtenerUsuario(posicion);

    if(usuario.getContrasenia().equals(contrasenia)){
        this.usuarioConectado = usuario;
        return true;
    }
    return false;

}
```

Y el apartado de la vistaConsola que se pide ingresar el rut, correo y la contraseña. Si los datos ingresados por el usuario son los mismos que los datos que registro el usuario se accederá a un menú donde podrá interactuar con el sistema de ofertas.

```
private void iniciarSesion() {  1 usage  ± ReimannHeiliger *
    String rut, contrasenia;

    StdOut.println(":::Iniciar Sesion:::");
    StdOut.println("Ingreses sus Credenciales");
    StdOut.println("Ingreses su rut:");
    rut = StdIn.readLine();
    StdOut.println("Ingreses su contrasenia:");
    contrasenia = StdIn.readLine();

    try {
        if (this.sistemaUsuario.iniciarSesion(rut, contrasenia)) {
            this.menuPrincipal();
        } else {
            StdOut.println("La contrasenia es invalida");
        }
    } catch (IllegalArgumentException e) {
        StdOut.println(e.getMessage());
    }
    StdOut.println();
}
```

Publicar Oferta

Para el método de ingresar ofertas hay 2 apartados: el apartado de SistemaOfertaArreglo que dependiendo del tipo de oferta se encarga de crear nuevas ofertas ya sea ayudantía, capstone y practica pre-profesional. Las validaciones que se hacen en general que incluye las ofertas de ayudantía, capstone y practica pre-profesional son de longitud de caracteres y de duración en días. Se implementaron 3 métodos para cada tipo de oferta, si los tipos de oferta son iguales al tipo de ofertas aceptados del arreglo retorna el método que se emplea para cada tipo de oferta.

```
    private final String[] CODIGOS_TIPO_OFERTA = {"A","C","P"};
@Override  1 usage  ReimannHeiliger *
public String publicarOferta(String[] datosOferta) {
    try {
        this.validarOferta(datosOferta);
    } catch (IllegalArgumentException e) {
        return e.getMessage();
    }
    // caso: ayudantia
    if (datosOferta[0].equalsIgnoreCase(this.CODIGOS_TIPO_OFERTA[0])) {
        return this.publicarOfertaAyudantia(datosOferta);
    }
    // caso: capstone
    if (datosOferta[0].equalsIgnoreCase(this.CODIGOS_TIPO_OFERTA[1])) {
        return this.publicarOfertaCapstone(datosOferta);
    }
    // caso: practica
    return this.publicarOfertaPractica(datosOferta);
}
```

1. **PublicarAyudantia:**

Se hacen validaciones de datos como de longitud de caracteres, cantidad de horas trabajadas semanalmente que no puede ser menor a 2 y del promedio que hay un rango de nota mínimo para que sea aceptado. Si se cumplen con los requisitos se agrega una nueva oferta de tipo ayudantía.

```
private String publicarOfertaAyudantia(String[] datosOferta) {  1 usage  ReimannHeiliger *
    try {
        this.validarAyudantia(datosOferta);
    } catch (IllegalArgumentException e) {
        return e.getMessage();
    }

    int ultimaID = this.contenedorNexoDoble.getTamanio()+1;//generador de id
    // se crea la oferta de ayudantia
    Oferta nueva = new OfertaAyudantia(ultimaID, datosOferta[1], datosOferta[2], datosOferta[3], Integer.parseInt(datosOferta[4]), datosOferta[5], datosOferta[6], Integer.parseInt(datosOferta[7]), Float.parseFloat(datosOferta[8]));
    // se agrega al listado
    this.contenedorNexoDoble.agregarOferta(nueva);
    // respuesta afirmativa
    return "Se ha agregado la oferta de ayudantia con exito, y la id es "+(this.contenedorNexoDoble.getTamanio());
}
```

2. PublicarCapstone:

Se hacen validaciones para los datos como el tipo de proyecto capstone, si es distinto a capstone, investigación o memoria significa que no es válido el tipo de proyecto ingresado, la duración en meses no puede ser distinta de 6, 12 o de 18 meses, longitud de caracteres que se debe cumplir un rango de caracteres para que sea válido y la cantidad mínima de estudiantes que debe ser entre 2 a 5 estudiantes. Si cumple con los requisitos se agrega una nueva oferta de tipo capstone.

```
private String publicarOfertaCapstone(String[] datosOferta) {  1 usage   ReimannHeiliger *
    try {
        this.validarCapstone(datosOferta);
    } catch (IllegalArgumentException e) {
        return e.getMessage();
    }
    int ultimaID = this.contenedorNexoDoble.getTamanio()+1;
    // se crea la oferta de capstone
    Oferta nueva = new OfertaCapstone(ultimaID, datosOferta[1], datosOferta[2], datosOferta[3], Integer.parseInt(datosOferta[4]), ValidadorFecha.convertirFecha(datosOferta[5]), datosOferta[6],
            datosOferta[7], datosOferta[8], Integer.parseInt(datosOferta[9]), this.obtenerCarreras(datosOferta), Integer.parseInt(datosOferta[11]));
    // se agrega la oferta a la lista
    this.contenedorNexoDoble.agregarOferta(nueva);

    return "La oferta de Capstone Project fue agregada con exito , y la id es "+(this.contenedorNexoDoble.getTamanio());
}
```

3. PublicarPracticaProfesional:

Se agrega una nueva oferta de tipo practica pre-profesional

```
private String publicarOfertaPractica(String[] datosOferta) {  1 usage   ReimannHeiliger *
    try {
        this.validarPractica(datosOferta);
    } catch (IllegalArgumentException e) {
        return e.getMessage();
    }
    int ultimaID = this.contenedorNexoDoble.getTamanio()+1;

    Oferta nueva = new OfertaAyudantia(ultimaID, datosOferta[1], datosOferta[2], datosOferta[3], Integer.parseInt(datosOferta[4]), datosOferta[5],
            datosOferta[6], Integer.parseInt(datosOferta[7]), Float.parseFloat(datosOferta[8]));
    // se agrega la oferta
    this.contenedorNexoDoble.agregarOferta(nueva);

    return "Se ha agregado la oferta con exito , y la id es "+(this.contenedorNexoDoble.getTamanio());
}
```

Y el apartado de la vistaConsola donde hay un menú indicando los tipos de ofertas disponibles y que dependiendo del tipo de oferta seleccionado se ingresan los datos asociados al tipo de oferta elegida creando métodos para cada una de las ofertas.

```
private void ingresarOferta() {  1 usage   ReimannHeiliger *
    // se sobredimensiona aqui, pero se sobreescribira luego
    String[] datosOferta = new String[99];

    StdOut.println(":::Ingresar Oferta:::");
    StdOut.println("Elija el tipo de oferta:");
    StdOut.println("1. Ayudantia");
    StdOut.println("2. Capstone");
    StdOut.println("3. Practica Pre-Profesional");
    int opcion = ValidadorEntrada.validarEntero( frase: "Ingrese una opcion: ");
```

Primero se ingresan los datos genéricos que tienen las 3 ofertas como ayudantía, capstone y práctica profesional:

```
private void ingresarDatosGenericos(String[] datosOferta) {  2 usages  ReimannHeiliger *
    // ingresar datos genericos
    StdOut.println("Ingrese el titulo de la oferta");
    datosOferta[1] = StdIn.readLine();

    StdOut.println("Ingrese el descripcion de la oferta");
    datosOferta[2] = StdIn.readLine();

    StdOut.println("Ingrese la unidad de la oferta");
    datosOferta[3] = StdIn.readLine();

    StdOut.println("Ingrese la duracion en dias de la oferta");
    datosOferta[4] = StdIn.readLine();
}
```

Si el tipo de oferta que se quiere ingresar es capstone y practica profesional se ingresan los siguientes datos:

```
private void ingresarDatosActividad(String[] datosOferta) {  2 usages  ReimannHeiliger *
    // ingresar datos de un capstone
    String dia, mes, anio;

    StdOut.println("Ingrese el dia de inicio de la actividad");
    dia = StdIn.readLine();

    StdOut.println("Ingrese el mes de inicio de la actividad");
    mes = StdIn.readLine();

    StdOut.println("Ingrese el anio de inicio de la actividad");
    anio = StdIn.readLine();

    datosOferta[5] = dia + "-" + mes + "-" + anio;

    StdOut.println("Ingrese el nombre de la empresa solicitante");
    datosOferta[6] = StdIn.readLine();

    StdOut.println("Ingrese el nombre del guia de la empresa");
    datosOferta[7] = StdIn.readLine();

}
```

IngresarDatosAyudantia:

En este método se ingresan los datos correspondientes a la oferta de tipo ayudantía. La id se genera automáticamente con ayuda del contenedor sumando 1 a la cantidad actual. Y finalmente se llama al método de ingresar ayudantía implementada en el sistema.

```
private void ingresarDatosAyudantia(String[] datosOferta) {  2 usages  Rei
    // ingresar datos de una ayudantia
    StdOut.println("Ingrese el nombre de la asignatura");
    datosOferta[5] = StdIn.readLine();

    StdOut.println("Ingrese el rol del ayudante");
    datosOferta[6] = StdIn.readLine();

    StdOut.println("Ingrese la cantidad de horas semanal mensual");
    datosOferta[7] = StdIn.readLine();

    StdOut.println("Ingrese el promedio minimo solicitado");
    datosOferta[8] = StdIn.readLine();

}
```

IngresarDatosCapstone:

En este método se ingresan los datos correspondientes a la oferta de tipo capstone. La id se genera automáticamente con ayuda del contenedor sumando 1 a la cantidad actual. Para los datos que sean números se hace uso de una clase llamada ValidadorEntrada que se encarga de manejar excepciones si se ingresan letras. Y finalmente se llama al método de ingresar capstone implementada en el sistema.

```
private void ingresarDatosCapstone(String[] datosOferta) {  2 usages  ReimannHeiliger *
    StdOut.println("Elija el tipo de capstone:");
    StdOut.println("1. Capstone");
    StdOut.println("2. Investigacion");
    StdOut.println("3. Memoria");

    int opcion = ValidadorEntrada.validarEntero( frase: "Ingrese una opcion: ");

    if (opcion == 1) {
        datosOferta[8] = "Capstone";
    }
    if (opcion == 2) {
        datosOferta[8] = "Investigacion";
    }
    if (opcion == 3) {
        datosOferta[8] = "Memoria";
    }
    StdOut.println();
```

```
        if (opcion < 1 || opcion > 3) {
            throw new InputMismatchException();
        }
        StdOut.println("Ingrese la duracion en meses del proyecto de capstone");
        datosOferta[9] = StdIn.readLine();
        StdOut.println();

        int cantidadCarreras = ValidadorEntrada.validarEntero( frase: "Ingrese la cantidad de carreras");
        datosOferta[10] = "";

        for (int i = 0; i < cantidadCarreras; i++) {
            StdOut.println("Ingrese el nombre de la carrera N°" + (i+1));
            datosOferta[10] += StdIn.readLine();
            if (i != cantidadCarreras - 1) {
                datosOferta[10] += ",";
            }
        }
        StdOut.println("Ingrese la cantidad de estudiantes requeridos para el proyecto de capstone");
        datosOferta[11] = StdIn.readLine();

    }
```

IngresarPracticaProfesional:

En este método se ingresan los datos correspondientes a la oferta de tipo practica pre-profesional. La id se genera automáticamente con ayuda del contenedor sumando 1 a la cantidad actual.

Para verificar si requiere de remuneración el proyecto capstone se pregunta si la requiere, si la respuesta es Si la será verdad que requiere remuneración, caso contrario si la respuesta es un no. Y finalmente se llama al método de ingresar ayudantía implementada en el sistema.

```
private void ingresarDatosPractica(String[] datosOferta) {  2 usages   ReimannHeiliger
    StdOut.println("1. Si");
    StdOut.println("2. No");
    int opcion = ValidadorEntrada.validarEntero( frase: "Tiene remuneracion la actividad de practica?");

    if (opcion == 1) {
        datosOferta[8] = "Si";
    } else if (opcion == 2) {
        datosOferta[8] = "No";
    } else {
        throw new InputMismatchException();
    }
}
```

Ver Oferta

Para este método hay 2 apartados: el apartado de SistemaOfertaArreglo que en base a la id se imprime por consola los datos de la oferta que está asociada a la id. Si la id está registrada en el sistema significa que efectivamente la oferta fue encontrada y dependiendo si la oferta es una instancia del tipo de oferta que está asociada a la id se imprimen los datos en base a la id ingresada por el usuario.

```
@Override  1 usage  ReimannHeiliger *
public String verOferta(String idTitulo) {//funcionando

    if (this.contenedorNexoDoble.getTamanio() == 0) {
        return "La lista esta vacia";
    }
    int id = -1;
    try {
        id = Integer.parseInt(idTitulo);
    } catch (NumberFormatException _) {
    }

    if (this.ultimaOferta != null && (this.ultimaOferta.getId() == id || this.ultimaOferta.getTitulo().equalsIgnoreCase(idTitulo))) {
        return this.ultimaOfertaInfo();
    }
    //obtiene la cabecera del nodo
    NodoDoble aux = this.contenedorNexoDoble.getCabecera();
    //validasi el nodo es nulo
    if(aux == null){
        return "La oferta no puede ser nula";
    }
    //while para recorrer lista
    while (aux != null){

        Oferta oferta = aux.getOferta();
        this.ultimaOferta = oferta;

        if(ultimaOferta.getId() == id || ultimaOferta.getTitulo().equals(idTitulo)){
            return this.ultimaOfertaInfo();
        }
        aux = aux.getSiguiente();
    }
    return "La oferta con el id: " + id + ", no existe.";
}

private String ultimaOfertaInfo() {  2 usages  ReimannHeiliger

    return ":::OFERTA ENCONTRADA:::" + "\n\n" +
            this.ultimaOferta + "\n\n";
}
```

Y el apartado de la vistaConsola que solo se llama al sistema que emplea el método correspondiente

```
private void buscarOferta() {  1 usage  ReimannHeiliger *
    String idTitulo;
    StdOut.println(":::Buscar Oferta:::");
    StdOut.println("Ingrese la id o titulo de la oferta");
    idTitulo = StdIn.readString();

    StdOut.println(this.sistemaOferta.verOferta(idTitulo));

}
```

VerTodasLasOfertas

Para este método hay 2 apartados: el apartado de SistemaOfertaArreglo que dependiendo del tipo de oferta se encarga de mostrar por pantalla los datos correspondientes de las ofertas ya sea ayudantía, capstone y practica pre-profesional usando el StringBuilder para crear un conjunto de Strings y usamos el nombre de la clase con toString para mostrar todos los datos.

```
@Override  1 usage  ReimannHeiliger *
public String verTodasLasOfertas() {

    if (this.contenedorNexoDoble.getTamanio() == 0) {
        return "La lista esta vacia";
    }

    StringBuilder sb = new StringBuilder();
    sb.append(":::OFERTAS DISPONIBLE:::").append("\n\n");

    NodoDoble aux = this.contenedorNexoDoble.getCabecera();

    while (aux != null){

        Oferta oferta = aux.getOferta();
        sb.append(oferta.toString());

        aux = aux.getSiguiente();
    }
    return sb.toString();
}
```

Y el apartado de la vistaConsola que solo se llama al sistema que emplea el método correspondiente

```
private void verOfertas() {  1 usage  ReimannHeiliger
    StdOut.println(":::Listado de Ofertas:::");
    StdOut.println(this.sistemaOferta.verTodasLasOfertas());

}
```

Editar Ofertas

Para este método hay 2 apartados: el apartado de SistemaOfertaArreglo que en base a la id se pueda editar los datos de la oferta que está asociada a la id. Si la id está registrada en el sistema significa que efectivamente la oferta fue encontrada y se podrá modificar los datos asociados al tipo de oferta en base a los métodos set() implementados en las clases tanto de la oferta como de los 3 tipos de ofertas como ayudantía, capstone y practica pre-profesional.

```java
@Override  1 usage  ReimannHeiliger *
public String editarOferta(String id, String[] datosOferta) {
    int id_validada;

    try {
        id_validada = Integer.parseInt(id);
    } catch (NumberFormatException e) {
        return "El id no corresponde a un numero.";
    }
    Oferta ofertaQueSeEdita = buscarOferta(id_validada);

    try {
        this.validarOferta(datosOferta);
    } catch (IllegalArgumentException e) {
        return e.getMessage();
    }
    // caso: ayudantia
    if (datosOferta[0].equalsIgnoreCase(this.CODIGOS_TIPO_OFERTA[0]) && ofertaQueSeEdita instanceof OfertaAyudantia) {
        return this.editarOfertaAyudantia(datosOferta, ofertaQueSeEdita);
    }
    // caso: capstone
    if (datosOferta[0].equalsIgnoreCase(this.CODIGOS_TIPO_OFERTA[1]) && ofertaQueSeEdita instanceof OfertaCapstone) {
        return this.editarOfertaCapstone(datosOferta, ofertaQueSeEdita);
    }
    // caso: practica
    if (ofertaQueSeEdita instanceof OfertaPractica) {
        return this.editarOfertaPractica(datosOferta, ofertaQueSeEdita);
    }
    return "La oferta con la id " + id + ", no corresponde con el tipo de oferta declarado";
}
```

Primero de inician editando los datos genéricos que comparten los 3 tipos de ofertas:

```java
// editores
private void editarSeccion(String[] datosOferta, Oferta oferta) {  3 usages  ReimannHeiliger
    // se edita la seccion que corresponde a la clase Oferta
    oferta.setTitulo(datosOferta[1]);
    oferta.setDescripcion(datosOferta[2]);
    oferta.setNombreUnidad(datosOferta[3]);
    oferta.asignarFechas(Integer.parseInt(datosOferta[4]));
}
```

1. **EditarOfertaAyudantia**

   Se ingresan los datos que se desean modificar para la oferta de ayudantía.

```
private void editarSeccion(String[] datosOferta, OfertaAyudantia oferta) {  1 usage  ReimannHeiliger
    // se edita la seccion que corresponde a la clase OfertaAyudantia
    oferta.setNombreAsignatura(datosOferta[5]);
    oferta.setRolAyudante(datosOferta[6]);
    oferta.setCantidadHorasSemanalMensual(Integer.parseInt(datosOferta[7]));
    oferta.setPromedioMinimo(Float.parseFloat(datosOferta[8]));
}
```

```
private String editarOfertaAyudantia(String[] datosOferta, Oferta oferta) {  1 usage  ReimannHeiliger *
    try {
        this.validarAyudantia(datosOferta);
    } catch (IllegalArgumentException e) {
        return e.getMessage();
    }
    // se edita la oferta
    this.editarSeccion(datosOferta, oferta);
    this.editarSeccion(datosOferta, (OfertaAyudantia) oferta);

    return "Se ha editado con exito la oferta de ayudantia con la id " + oferta.getId();
}
```

2. **EditarOfertaCapstone:**

   Se ingresan los datos que se desean modificar para la oferta de capstone.

```
private void editarSeccion(String[] datosOferta, OfertaCapstone oferta) {  1 usage  ReimannHeiliger
    // se edita la seccion que corresponde a la clase OfertaCapstone
    oferta.setFechaInicio(ValidadorFecha.convertirFecha(datosOferta[5]));
    oferta.setEmpresaSolicitante(datosOferta[6]);
    oferta.setGuiaEmpresa(datosOferta[7]);
    oferta.setTipoCapstone(datosOferta[8]);
    oferta.setDuracionEnMeses(Integer.parseInt(datosOferta[9]));
    oferta.setCarrerasNecesarias(this.obtenerCarreras(datosOferta));
    oferta.setCantidadMinimaEstudiantes(Integer.parseInt(datosOferta[11]));
}
```

```
private String editarOfertaCapstone(String[] datosOferta, Oferta oferta) {  1 usage  ReimannHeiliger *
    try {
        this.validarCapstone(datosOferta);
    } catch (IllegalArgumentException e) {
        return e.getMessage();
    }

    // se edita la oferta
    this.editarSeccion(datosOferta, oferta);
    this.editarSeccion(datosOferta, (OfertaCapstone) oferta);

    return "Se ha editado la oferta de capstone con la id " + oferta.getId();
}
```

3. EditarOfertaPractica:

Se ingresan los datos que se desean modificar para la oferta de práctica profesional

```
private void editarSeccion(String[] datosOferta, OfertaPractica oferta) {  1 usage  ReimannHeiliger
    // se edita la seccion que corresponde a la clase OfertaPractica
    oferta.setFechaInicio(ValidadorFecha.convertirFecha(datosOferta[5]));
    oferta.setEmpresaSolicitante(datosOferta[6]);
    oferta.setGuiaEmpresa(datosOferta[7]);
    oferta.setTieneRemuneracion(datosOferta[8].equalsIgnoreCase( anotherString: "Si"));
}


private String editarOfertaPractica(String[] datosOferta, Oferta oferta) {  1 usage  ReimannHeiliger *
    try {
        this.validarPractica(datosOferta);
    } catch (IllegalArgumentException e) {
        return e.getMessage();
    }

    // se edita la oferta
    this.editarSeccion(datosOferta, oferta);
    this.editarSeccion(datosOferta, (OfertaPractica) oferta);

    return "Se ha editado la oferta de practica profesional con la id " + oferta.getId();
}
```

Y el apartado de la VistaConsola donde se pide por pantalla la id de la oferta que se desea editar. Si la id fue encontrada se ingresan los datos iniciales que tienen los 3 tipos de ofertas y dependiendo si la id de la oferta corresponde a un tipo de oferta en específico se ingresan los datos adicionales que corresponde al tipo de oferta identificada por la id.

```
private void editarOferta() {  1 usage  ReimannHeiliger *
    // se sobredimensiona aqui, pero se sobreescribira luego
    String[] datosOferta = new String[99];
    StdOut.println(":::Editar Oferta:::");
    StdOut.println("Ingrese el id de la oferta");
    String id = StdIn.readLine();
    StdOut.println("Elija el tipo de oferta:");
    StdOut.println("1. Ayudantia");
    StdOut.println("2. Capstone");
    StdOut.println("3. Practica Pre-Profesional");
    int opcion = ValidadorEntrada.validarEntero( frase: "Ingrese una opcion: ");
```

```
    if (opcion == 1) {
        datosOferta = new String[9];
        datosOferta[0] = "A";
        StdOut.println(":::Ingresar Ayudantia:::");
    }
    if (opcion == 2) {
        datosOferta = new String[12];
        datosOferta[0] = "C";
        StdOut.println(":::Ingresar Capstone:::");
    }
    if (opcion == 3) {
        datosOferta = new String[9];
        datosOferta[0] = "P";
        StdOut.println(":::Ingresar Practica:::");
    }
    if (opcion < 1 || opcion > 3) {
        StdOut.println("opcion invalida. volviendo al menu.");
        return;
    }
    this.ingresarDatosGenericos(datosOferta);
    try {
        if (opcion == 1) {
            this.ingresarDatosAyudantia(datosOferta);
        } else {
            this.ingresarDatosActividad(datosOferta);

            if (opcion == 2) {
                this.ingresarDatosCapstone(datosOferta);
            } else {
                this.ingresarDatosPractica(datosOferta);
            }
        }
    } catch (InputMismatchException e) {
        StdOut.println("opcion invalida. volviendo al menu.");
        return;
    }
    StdOut.println(this.sistemaOferta.editarOferta(id, datosOferta));
}
```

Eliminar Oferta

Para este método hay 2 apartados: el apartado de SistemaUsuarioArreglo que en base a la id se pueda eliminar una oferta del sistema.

```java
@Override  1 usage  ReimannHeiliger *
public String eliminarOferta(String id) {

    int idOferta;
    try {
        idOferta = Integer.parseInt(id);
    } catch (NumberFormatException e) {
        return "El id no corresponde a un numero.";
    }
    NodoDoble aux = this.contenedorNexoDoble.getCabecera();

    while (aux != null){

        Oferta oferta = aux.getOferta();

        if(oferta.getId() == idOferta){//si la id asociada a la oferta es igual a la id dada por parametro
            this.contenedorNexoDoble.eliminar(oferta);//se elimina la oferta

            if(this.ultimaOferta != null && this.ultimaOferta == oferta){
                this.ultimaOferta = null;//la oferta asociada se hace nula porque la oferta dejara de existir
            }
            return "Se ha eliminado la oferta con la id "+id+ "con exito";
        }
        aux = aux.getSiguiente();//pasa al siguinte
    }
    return "La oferta con el id: " + id + ", no existe.";
}
```

Y el apartado de la VistaConsola que pide ingresar la id que se desea eliminar. Una vez encontrada la id se vuelve a preguntar para confirmar si está seguro de eliminar la oferta. Si la respuesta es sí la oferta se elimina y si ocurre lo contrario la eliminación de la oferta será cancelada. Y por último, se pregunta si se desean seguir eliminando ofertas, si la respuesta es sí repite el proceso, y si la respuesta es no, se interrumpe el proceso regresando al menú.

```java
private void eliminarOferta() {  1 usage  ReimannHeiliger
    StdOut.println(":::Eliminar Oferta:::");

    StdOut.println("Ingrese el id de la oferta");
    String id = StdIn.readLine();

    StdOut.println("Esta seguro de eliminar la oferta con la id: " + id);
    StdOut.println("1. Si");
    StdOut.println("2. No");
    int opcion = ValidadorEntrada.validarEntero( frase: "Ingrese una opcion: ");

    if (opcion == 1) {
        StdOut.println(this.sistemaOferta.eliminarOferta(id));
    } else {
        StdOut.println("Volviendo al menu.");
    }
}
```

Ver Perfil

En este método hay 2 apartados: el apartado de SistemaUsuarioArreglo que imprime los datos del usuario como el nombre, rut y el correo a excepción de la contraseña que nunca debe mostrarse por pantalla.

```
@Override  1 usage  ReimannHeiliger *
public String verPerfil() {

    Nodo aux = this.contenedorNexoSimple.getCabecera();

    if(aux == null){
        return "el usuario no puede ser nulo";
    }
    this.usuarioConectado = aux.getUsuario();

    if(this.usuarioConectado == null){
        return "no hay personas en sistema";
    }
    return this.usuarioConectado.toString();
}
```

Y el apartado de la vistaConsola que solo llama al sistema que emplea el método correspondiente.

```
private void verPerfil() {  1 usage  ReimannHeiliger
    StdOut.println(":::Perfil:::");
    StdOut.println(this.sistemaUsuario.verPerfil());

}
```

Editar Perfil

Para este método hay 2 apartados: el apartado del SistemaUsuarioArreglo donde se puede editar la información el usuario como el nombre y el correo. Para implementar este método usamos la clase Usuario usuarioConectado que instanciamos al inicio. Si el usuario conectado es nulo, significa que no hay usuarios registrados, si el correo del usuario conectado es igual al nuevo correo significa que el nuevo correo no puede ser igual al correo actual.

Luego si el usuario es distinto a nulo y si el correo es igual al nuevo correo quiere decir que el correo ya está en uso por lo que sí o sí debe ingresar un correo único. Y finalmente se modifica el nombre y correo del usuario que está conectado.

```java
@Override  ReimannHeiliger *
public String editarPerfil(String nombre, String correo) {

    if (nombre.length() < 3 || nombre.length() > 255) {
        return "El nombre no puede tener menos de 3 caracteres o mas de 255.";
    }
    if (this.usuarioConectado == null) {
        return "No hay usuario conectado";
    }
    Nodo aux = this.contenedorNexoSimple.getCabecera();

    if(aux == null){
        return "El ususario no puede ser nulo";
    }

    while (aux != null){

        if(aux.getUsuario() == this.usuarioConectado){//compara el objeto directamente

            this.usuarioConectado.setCorreo(correo);
            this.usuarioConectado.setNombre(nombre);
            return "se ha editado el perfil con exito";
        }
        aux = aux.getSiguiente();
    }
    return "Usuario no encontrado en la lista";
}
```

Y el apartado de la vistaConsola que pide por pantalla ingresar solamente el nombre y el correo nuevo que se quiere editar, se valida la longitud de caracteres del nuevo nombre que debe cumplir con un mínimo de caracteres para que sea aceptado, después se pide ingresar el correo.

Una vez finalizado se llama al sistema que emplea el método correspondiente terminando el proceso. Y finalmente se pregunta si se desea seguir editando el perfil del usuario, si la respuesta es sí continua con la edición del perfil del usuario, caso contrario que si la respuesta es no, termina de editar y regresa al menú.

```java
private void editarPerfil() {  1 usage  ReimannHeiliger
    String nombre, correo;

    StdOut.println(":::Editar Perfil:::");
    StdOut.println("Ingrese su nombre:");
    nombre = StdIn.readLine();
    StdOut.println("Ingrese su correo:");
    correo = StdIn.readLine();

    StdOut.println(this.sistemaUsuario.editarPerfil(nombre, correo));

}
```

Salir:

Una vez realizadas las operaciones utilizadas por el usuario al ingresar la opción de salir se va a terminar la ejecución del programa e invocara a los métodos asociados a las persistencias tanto de las ofertas como de usuarios cuyos métodos emplean archivos de salida escribiendo los datos obtenidos en los métodos del programa.

```
if (opcion == 3) {
    try {
        this.sistemaUsuario.persistenciaUsuarios();
        this.sistemaOferta.persistenciaOfertas();
    } catch (RuntimeException e) {
        StdOut.println(e.getMessage());
    }
    break;
}
```

PersistenciaUsuarios:

Invoca al método de escribir archivo el cual su función es crear un archivo de salida. Como los usuarios se trabajan con listas con nexo simples se llama a la cabecera y se recorre la lista para escribir los datos.

```
@Override  1 usage  ReimannHeiliger *
public void persistenciaUsuarios() {

    try {
        this.escribirArchivo();
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}

private void escribirArchivo() throws IOException {  1 usage  new *

    ArchivoSalida archivoSalida = new ArchivoSalida( nombreSalida: "Resultados_usuario.txt");

    Nodo aux = this.contenedorNexoSimple.getCabecera();

    while (aux != null){

        Usuario usuario = aux.getUsuario();
        Registro regSal = new Registro( cantidadCampos: 4);

        regSal.agregarCampo(usuario.getNombre());
        regSal.agregarCampo(usuario.getRut());
        regSal.agregarCampo(usuario.getCorreo());
        regSal.agregarCampo(usuario.getContrasenia());

        archivoSalida.writeRegistro(regSal);
        aux = aux.getSiguiente();
    }
    archivoSalida.close();
}
```

PersistenciaOfertas:

Se llama al método escribir archivo el cual se encarga de crear el archivo de salida. Caso muy similar se recorre la lista de la misma manera. Se llama a la cabecera y usa SimpleDateFormat el cual se encarga de establecer un formato para las fechas.

```java
@Override  1 usage  ReimannHeiliger *
public void persistenciaOfertas() {

    try {
        this.escribirArchivo();
    } catch (IOException e) {
        StdOut.println(e.getMessage());
    }
}
```

```java
private void escribirArchivo() throws IOException {  1 usage  new *

    ArchivoSalida archivoSalida = new ArchivoSalida( nombreSalida: "Resultados_academicos.txt");

    NodoDoble aux = this.contenedorNexoDoble.getCabecera();
    //formato de fecha basado en dia, mes y anio
    SimpleDateFormat dateFormat = new SimpleDateFormat( pattern: "dd-MM-yyyy");

    while (aux != null) {

        Oferta oferta = aux.getOferta();

        if (oferta instanceof OfertaAyudantia) {
            Registro regSalAyudantia = new Registro( cantidadCampos: 10);
            String tipoOferta = "A";

            regSalAyudantia.agregarCampo(tipoOferta);
            regSalAyudantia.agregarCampo(oferta.getId());
            regSalAyudantia.agregarCampo(oferta.getTitulo());
            regSalAyudantia.agregarCampo(oferta.getDescripcion());
            regSalAyudantia.agregarCampo(oferta.getNombreUnidad());

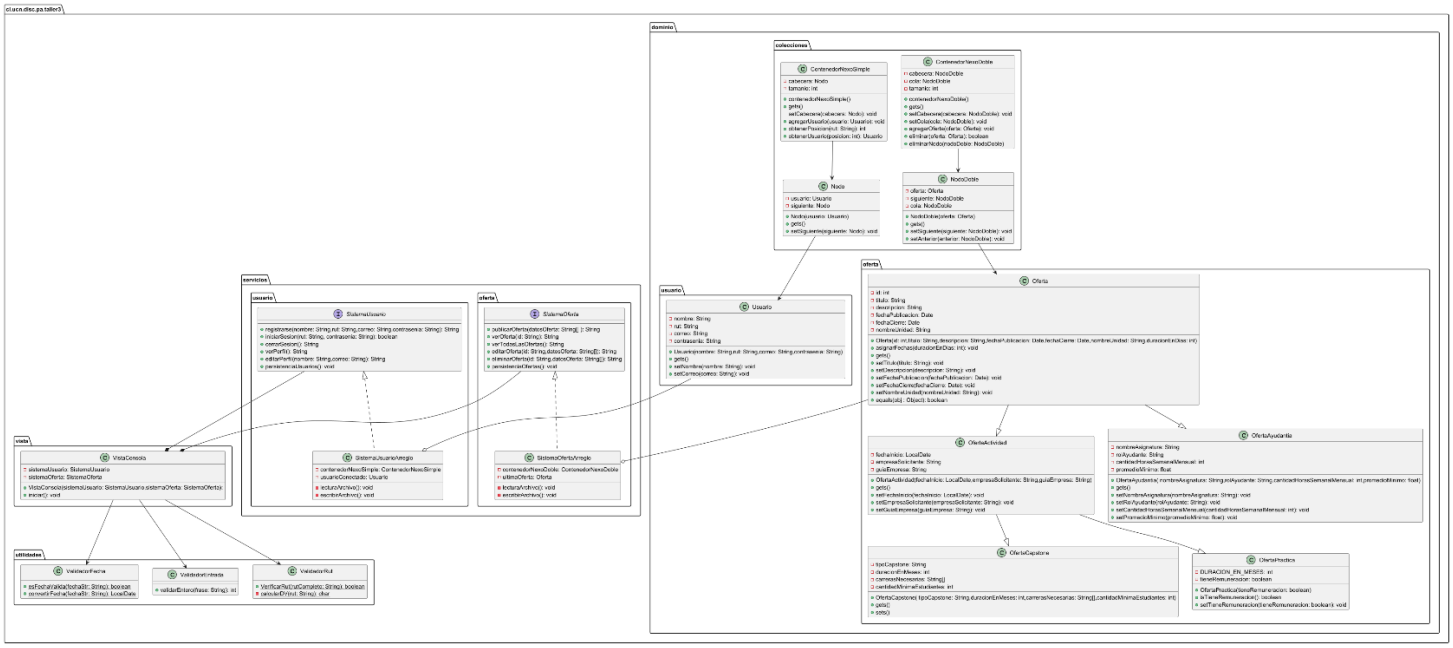
            // --- Corrección: Manejo de fecha de publicación nula para OfertaAyudantia ---
            Date fechaPublicacionAyudantia = oferta.getFechaPublicacion();
            if (fechaPublicacionAyudantia != null) {
                regSalAyudantia.agregarCampo(dateFormat.format(fechaPublicacionAyudantia));
            } else {
                regSalAyudantia.agregarCampo( datoString: ""); // Agrega cadena vacía si la fecha es nula
            }

            regSalAyudantia.agregarCampo(((OfertaAyudantia)oferta).getNombreAsignatura());
            regSalAyudantia.agregarCampo(((OfertaAyudantia)oferta).getRolAyudante());
            regSalAyudantia.agregarCampo(((OfertaAyudantia)oferta).getCantidadHorasSemanalMensual());
            regSalAyudantia.agregarCampo(((OfertaAyudantia)oferta).getPromedioMinimo());
```

Diagrama de Clase

Para la elaboración del diagrama de clase utilizamos una estructura de paquetes los cuales son:

- Dominio: contiene las clases que representan al usuario como las ofertas, y los contenedores tanto del usuario como de la oferta.
- Colecciones: contiene clases nodo y nodoDoble con sus respectivos contenedores tanto de lista con nexo simple como doble.
- Servicios: contiene la infertace sistema y la clase que implementa el sistema.
- Utilidades: clases que validan datos de entrada como números, promedios y rut.
- Vista: contiene la vista de la consola que interactúa con el usuario.



Contratos

Contrato 1

Operación	registrarse(nombre: cadena de texto, correo: cadena de texto, contraseña: cadena de texto)
Descripción	Registrar a un usuario al sistema pidiendo la siguiente información, nombre, correo, contraseña.
Precondición	El Rut debe ser validado, el nombre debe tener de 3 a 255 caracteres, la contraseña debe tener 6 a 255 caracteres, y no puede haber usuarios con el mismo Rut.
Postcondición	Agregar nuevos usuarios

Contrato 2

Operación	iniciarSesion(Rut: cadena de texto, correo: cadena de texto, contraseña: cadena de texto)
Descripción	Iniciar la sesión en el sistema con la validación de su Rut y su contraseña.
Precondición	El Rut debe ser validado y el usuario debe estar registrado al sistema.
Postcondición	Se ingresa al sistema pasando a ser el usuario conectado.

Contrato 3

Operación	ingresarOferta(id: cadena de texto, título: cadena de texto, descripción: cadena de texto, duración en días: cadena de texto)
Descripción	Creación de nuevas ofertas, pidiéndole al usuario que tipo de oferta es, es decir, ayudantía, capstone, práctica pre-profecional.
Precondición	Crea un código único por cada oferta y debe asegurarse de que todos los datos se hayan completados.
Postcondición	Se pude modifica o eliminar una oferta del usuario.

Contrato 4

Operación	buscarOferta(id: int, título: cadena de texto)
Descripción	Muestra todos los datos relevantes de la oferta, incluyendo su tipo y los datos particulares asociados a dicho tipo.
Precondición	Para mostrar la oferta se debe ingresar su código único o su título y el usuario debe tener una oferta.
Postcondición	No aplica

Contrato 5

Operación	verOferta()
Descripción	Muestra los datos de la oferta, por ejemplo, Rut, titulo, descripción y duración de días.
Precondición	El usuario debe tener una oferta.
Postcondición	No aplica.

Contrato 6

Operación	editarOferta(id: int)
Descripción	El usuario ingresará la ID de la oferta, tendrá que ingresar todos los datos necesarios de la oferta.
Precondición	Se debe validar todos los datos de la oferta.
Postcondición	No aplica.

Contrato 7

Operación	eliminarOferta(id int)
Descripción	Para eliminar la oferta se debe ingresar su ID, para que se pueda localizar la oferta.
Precondición	Se debe notificar una confirmación al usuario de si realmente quiere eliminar la oferta
Postcondición	No aplica

Contrato 8

Operación	verPerfil()
Descripción	Muestra los datos no sensibles del usuario, por ejemplo, nombre, Rut, correo.
Precondición	Debe estar registrado el usuario en el sistema y no se mostrarán datos importantes, es decir, la contraseña.
Postcondición	No aplica.

Contrato 9

Operación	editarPerfil(Rut: cadena de texto, correo: cadena de texto, contraseña: cadena de texto)
Descripción	El sistema solicitará al usuario ingresar su nombre y su correo que se desea modificar
Precondición	Los datos serán validados antes de actualizarlos
Postcondición	No aplica

Contrato 10

Operación	validarRut(Rut: cadena de texto)
Descripción	Se validará que el Rut ingresado por el usuario sea válido para el ingreso del sistema
Precondición	El Rut debe tener 8 dígitos y un dígito verificador calculado matemáticamente y no debe tener caracteres no permitidos ni espacios.
PostCondición	No aplica